Open in Colab

Variables

In Python variables are the names of the objects given by the programmer.

Assignment Statements

To give a name to a Python object you can do it with `=` at the begining of a new line.

In [1]:
```
greeting = 'Hello, world!'
```

Now the string `'Hello, world!'` has the name `greeting`. You can use `greeting` whenever you want to use that object.

In [2]:
```
greeting
```

Out[2]:
```
'Hello, world!'
```

In [3]:
```
greeting * 3
```

Out[3]:
```
'Hello, world!Hello, world!Hello, world!'
```

You can give multiple names to the same object.

In [4]:
```
greeting_message = message = greeting
```

Now the object `Hello, world!` has three names: `greeting`, `message` and `greeting_message`. All are the same object. You can see this by asking with `is` keyword:

In [5]:
```
greeting is greeting_message
```

Out[5]:
```
True
```

Warning: here the keyword `is` does not do an equality check. It says wheather two variables refer to the same object or not.

For example, if you write:

In [6]:
```
greeting_message = 'Hello, world!'
```

Then `greeting is greeting_message` becomes `False`

In [7]:
```
greeting is greeting_message
```

Out[7]:
```
False
```

Because these are now refering to two different objects.

Garbage Collection

Python objects remain in memory as soon as thay are refered by a variable. If for some reason thay lost their references they are automatically removed from the memory by Python. This is

called *garbage collection*. You can delete a variable by `del` keyword.

In [8]:
```
del greeting
```

The variable `greeting` is deleted but since the object referred by `greeting` still referred by another variable `message`, it is still in the memory.

In [9]:
```
del message
```

Now it is deleted from memory.

If you re-asign another Python object to a variable then the previously refered object will be garbage collected if there is no other reference to that object.

Naming Rules

In Python variable names contains letters, numbers and `_`. But they can not start with a number. For example `1a` can not be a variable name.

Also the following names can not be used as variable names:

`False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `try`, `while`, `with`, `yield`

Because they have special meanings.

- Using meaninful names for variables help people to understand your code.
- Usually UPPERCASE variable names are used for constants.
- When the variable name has more than one word use `_` to seperate them.

In [10]:
```
PI = 3.14
greeting_message = "Hello, world!"
```

Compound Assignment Statements

The following shortcuts are handy:

shortcut	meaning
`a += b`	`a = a + b`
`a -= b`	`a = a - b`
`a *= b`	`a = a * b`
`a**= b`	`a = a** b`
`a /= b`	`a = a / b`
`a//= b`	`a = a// b`
`a %= b`	`a = a % b`
`a &= b`	`a = a & b`
`a |= b`	`a = a | b`

shortcut	meaning
`a ^= b`	`a = a ^ b`

In [11]:
```
a = 12
a += 1
a
```

Out[11]: 13

Walrus Operator

In Python assignment statements must be done in a seperate line. But with Walrus operator `:=` you can assign values to the variables inside an expression.

In [12]:
```
2**(n := 32) - n
```

Out[12]: 4294967264

This is equivalent to:

In [13]:
```
n = 32
2**n - n
```

Out[13]: 4294967264

Therefore you can do variable assignment at first use, simplifying the code.